

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-1: INTRODUCTION TO UNITY & VR DEVELOPMENT

Aim:

To install Unity Hub and Unity 2022.3.62f3 LTS, create the first Unity project, and understand the Unity Editor.

Theory:

Unity is a real-time game engine used for creating 2D, 3D, AR and VR applications. Before developing VR projects, students should first learn how to install Unity, create a project and understand the Unity Editor.

Procedure:

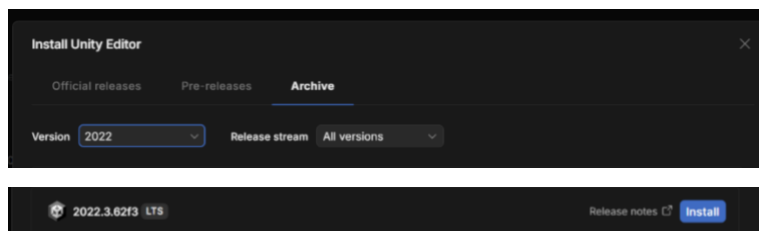
Step-1: Download Unity Hub from <https://unity.com/> download.

Step-2: Install Unity Hub.

Step-3: Sign in with a Unity account.

Step-4: Go to Installs → Install Editor.

Step-5: Archive, Switch to 2022 Version and Select Unity 2022.3.62f3 LTS.

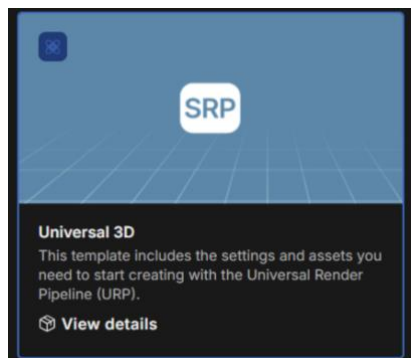


Step-6: Install Unity 2022.3.62f3 LTS and Microsoft Visual Studio.

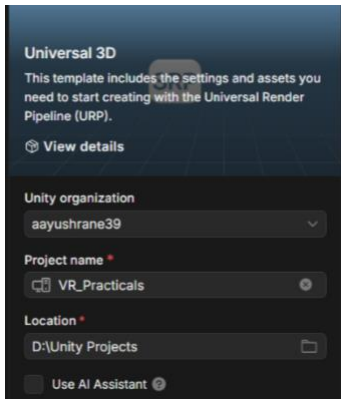


Step-7: Open Unity Hub and click New Project.

Step-8: Choose 3D (Core).



1. **Step-9:** Name the project 'VR_Practicals'.

TYB.Sc (I.T.) Semester V

Step-11: Click Create Project.

Step-12: Observe the Hierarchy, Scene, Game, Inspector, Project and Console windows.

Step-13: Save the scene as MainScene.

Output:

Unity Hub and Unity 2022.3.62f1 LTS installed successfully. A new 3D Unity project named 'VR Practicals' is created and the MainScene is saved.



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-2: Designing VR UI

Aim:

To design a **Puzzle Room game menu interface using Canva** with a suitable title, tagline, and menu options.

Theory

Canva is a graphic design platform that can be used to create attractive game interfaces and visual layouts. It provides text, shapes, images, and design elements that can be arranged to create a professional interface. A properly organized game menu improves the visual appearance and user experience.

Procedure

Step-1: Create Design: Open Canva and create a new design with a suitable screen size for the Puzzle Room game interface.

Step-2: Set Background: Add a suitable background to create the required puzzle-room atmosphere.

Step-3: Add Game Title: Add the title **PUZZLE ROOM** at the top of the design.

Step-4: Add Tagline: Add the tagline **ESCAPE THE MYSTERY** below the title and place decorative lines around it.

Step-5: Create Menu Area: Create a separate section in the centre of the design for the game menu options.

Step-6: Add Start Button: Add a button labelled **START GAME** to allow the player to begin the game.

Step-7: Add Instructions Button: Add a button labelled **INSTRUCTIONS** for displaying the game instructions.

Step-8: Add Exit Button: Add a button labelled **EXIT GAME** for exiting the game.

Step-9: Apply Styling: Adjust the fonts, sizes, spacing, shapes, and other visual elements to make the menu look consistent and attractive. The original design uses **Bebas Neue** and **Orbitron** style fonts.

Step-10: Finalize Design: Align all elements properly and review the complete Puzzle Room menu to ensure that the title, tagline, and three menu options are clearly visible.

Step-11: Export Design: Save or export the completed Puzzle Room interface in the required image format.

PUZZLE ROOM

— ESCAPE THE MYSTERY —

START GAME

INSTRUCTIONS

EXIT GAME

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-3-: Building a 3D Environment

Aim:

To create and design a basic **3D environment using Unity** by adding and arranging 3D objects, applying materials, setting up lighting, and creating a suitable scene.

Theory:

Unity is a game development engine used to create interactive 2D and 3D applications. A 3D environment is created by arranging GameObjects, applying materials, and setting up lighting and cameras. Unity allows objects to be positioned, rotated, and scaled to create the required virtual environment.

Procedure:

Step-1: Create Project: Open Unity Hub and create a new 3D project.

Step-2: Create Scene: Open the newly created project and create or open a scene.

Step-3: Add Objects: Add the required 3D objects such as planes, cubes, spheres, or other GameObjects.

Step-4: Arrange Objects: Adjust the position, rotation, and scale of the objects using the Transform component.

Step-5: Apply Materials: Create or select suitable materials and apply them to the objects.

Step-6: Set Lighting: Add and configure suitable light sources to illuminate the 3D environment.

Step-7: Set Camera: Position the Main Camera to obtain a proper view of the complete environment.

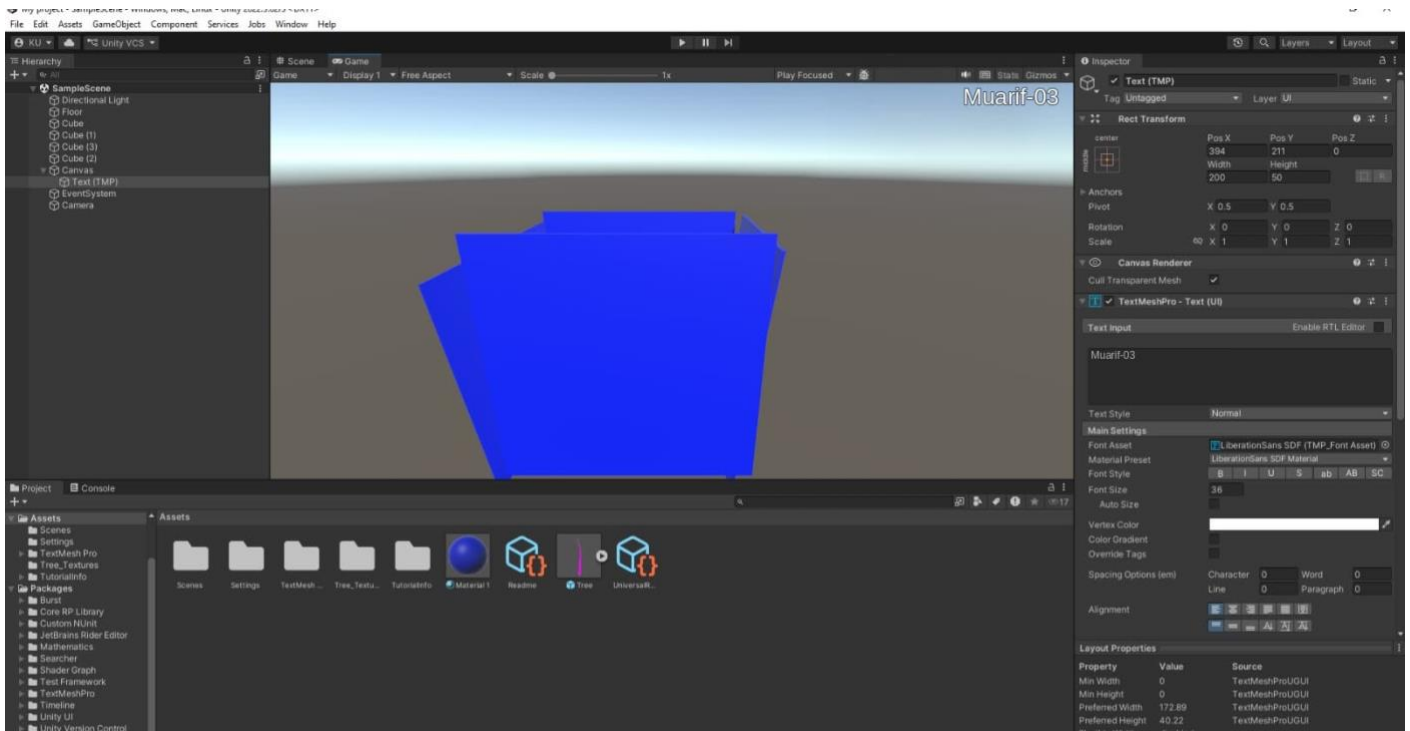
Step-8: Save Scene: Save the completed scene and project.

Step-9: Click the Play button to run the scene and verify that the 3D environment is displayed correctly.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Output:



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V PRACTICAL-4 -: Importing Assets

Aim:

To create a basic **3D puzzle room environment in Unity** using the Low Poly 3D Dungeon asset and by arranging and modifying the required dungeon components.

Theory:

Unity provides a 3D environment where pre-built assets can be imported and arranged to create interactive scenes. The Low Poly 3D Dungeon asset provides ready-made walls, pillars, materials, and scenes. These components can be modified and arranged in Unity to create a customized puzzle room.

Procedure:

Step-1: Add Assets: Go to **Windows** → **Asset Store**, search for **Dungeon**, select the free **Low Poly 3D**

Step-2: Configure Materials: Open the folder containing the imported packages, go to the **Material** folder, select all the materials, and go to **Edit** → **Rendering** → **Materials**. Select the option containing **URP**.

Step-3: Save the Scene: Select the **Untitled** scene and use **Save Scene As** to save it with the name **Puzzle Room**.

Step-4: Open Low Poly Dungeon: Go to **Low Poly Dungeon** → **Scene** and select the required dungeon scene.

Step-5: Add Dungeon to Hierarchy: Drag the selected dungeon scene into the **Hierarchy** window.

Step-7: Place Dungeon in Puzzle Room: Select **Dungeon of the Low Poly** and drag it into the **Puzzle Room** scene.

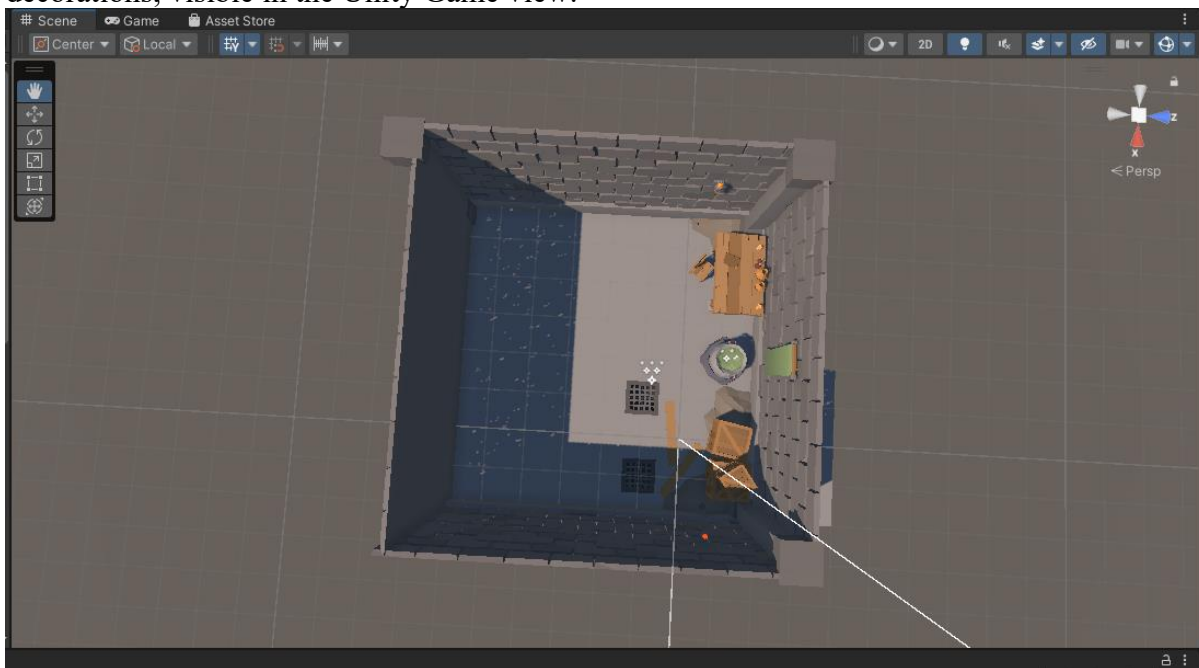
Step-8: Remove Original Scene: Select the **Low Poly** scene, click the three-dot menu, select **Remove Scene**, and choose not to save it.

Step-9: Duplicate the Wall: Select one side of the wall along with the two pillars, press **Ctrl + D**, and drag using the blue line to position the duplicate and create the fourth wall.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Output: A fully rendered, lit 3D dungeon room populated with walls, flooring, a doorway, and library decorations, visible in the Unity Game view.



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-5: XR Device Simulator

Aim:

To set up the **XR Interaction Toolkit** and **XR Device Simulator** in Unity and configure the project for basic XR interaction.

Theory:

The XR Interaction Toolkit is a Unity package used to create interactive VR and AR experiences. It provides components for XR controllers, interaction systems, and input handling. The XR Device Simulator allows XR interactions to be tested within the Unity Editor without requiring a physical XR device.

Procedure:

Step-1: Install XR Interaction Toolkit: Open **Window** → **Package Manager**, select **Unity Registry**, search for **XR Interaction Toolkit**, install the package, select **Yes** when prompted to restart, and save the project.

Step-2: Import Starter Assets: Open **Package Manager** → **XR Interaction Toolkit** → **Samples** and import **Starter Assets** and **XR Device Simulator**.

Step-3: Add XR Origin: Remove the existing **Main Camera** from the Hierarchy and drag the **XR Origin** into the Hierarchy window.

Step-4: Add XR Device Simulator: Add the **XR Device Simulator** to the Hierarchy window.

Step-5: Configure Input Action Manager: Add/configure the **Input Action Manager** required for handling XR input actions.

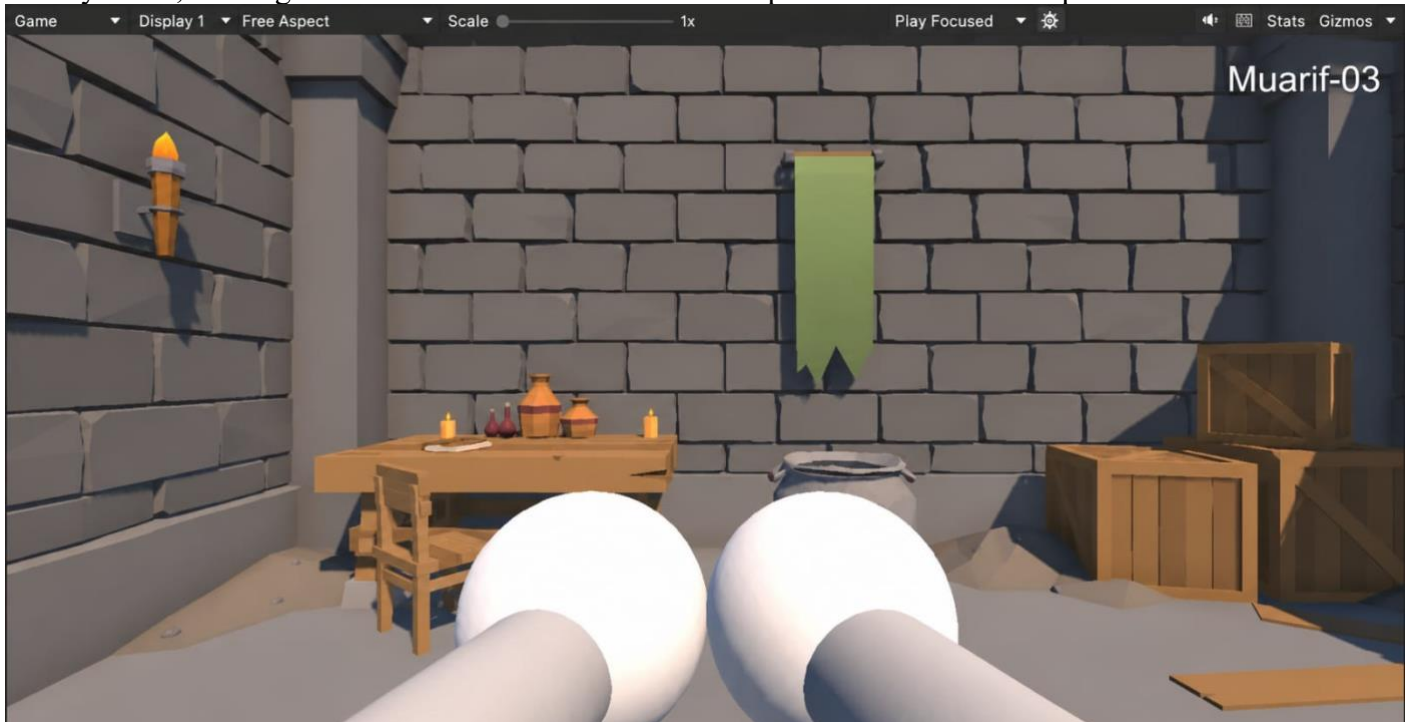
Step-6: Verify XR Components: Check the Hierarchy and ensure that the **XR Device Simulator** and required XR components are present.

Step-7: Run the Project: Click the **Play** button to run the game and test the XR environment using the simulator.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Output: The Hierarchy window displays the XR Interaction Manager, XR Device Simulator, and XR Origin. In Play mode, floating virtual controllers are visible and responsive to simulator inputs.



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-6: VR Locomotion

Aim:

To configure **XR locomotion** in Unity by adding the XR Locomotion System and Continuous Move Provider to enable movement in the XR environment.

Theory:

XR locomotion allows users to move around within a virtual environment using XR controllers. Unity's XR Interaction Toolkit provides locomotion components that can be configured for different movement methods. The Continuous Move Provider allows continuous movement based on the configured input actions.

Procedure:

Step-1: Install XR Interaction Toolkit: Go to **Window → Package Manager**, select **Unity Registry**, search for **XR Interaction Toolkit**, install it, restart Unity if prompted, and save the project.

Step-2: Import Starter Assets: Open **XR Interaction Toolkit → Samples** and import **Starter Assets** and **XR Device Simulator**.

Step-3: Add XR Origin: Remove the **Main Camera** and drag the **XR Origin** into the Hierarchy window.

Step-4: Add XR Simulator: Add the **XR Device Simulator** to the Hierarchy.

Step-5: Add Input Action Manager: Add the **Input Action Manager** and ensure the XR Device Simulator is present in the Hierarchy.

Step-6: Run the Project: Click the **Play** button to run the XR environment.

Step-7: Add Locomotion System: Go to **GameObject → XR → Locomotion System** and add the XR Locomotion System to the scene.

Step-8: Add Continuous Movement: Select the XR Locomotion System and add the **Continuous Move Provider (Action-based)** component.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Output: The Hierarchy window displays the XR Interaction Manager, XR Device Simulator, and XR Origin. In Play mode, floating virtual controllers are visible and responsive to simulator inputs.



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V PRACTICAL-7: Object Interaction

Aim:

To create an interactive **XR puzzle room environment** in Unity and test object interaction using the XR Device Simulator.

Theory

Unity XR Interaction Toolkit allows users to interact with virtual objects in a VR environment. Interactive objects such as gems, treasure chests, and other game elements can be placed inside a scene and tested using the XR Device Simulator. This helps in developing and testing VR applications without requiring a physical VR headset.

Procedure

Step-1: Open the Puzzle Room Scene: Open the previously created Puzzle Room scene containing the XR setup and dungeon environment.

Step-2: Add Interactive Objects: Place the required game objects such as the treasure chest, gems, and other puzzle elements inside the scene.

Step-3: Arrange the Environment: Position the objects properly within the room and adjust their scale and placement as required.

Step-4: Configure XR Interaction: Ensure that the XR Origin, XR Device Simulator, and interaction components are properly configured.

Step-5: Test the Scene: Click the **Play** button and use the XR Device Simulator to interact with the objects inside the puzzle room.

Code:- (GemSpawnManager.cs)

```
using UnityEngine;

public class GemSpawnManager : MonoBehaviour
{
    public GameObject[] Gems;
    public Transform[] SpawnPoints;

    void Start()
    {
        SpawnGems();
    }

    void SpawnGems()
    {
        // Create a list of available spawn points
```

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

System.Collections.Generic.List<Transform> availablePoints =

new System.Collections.Generic.List<Transform>(SpawnPoints);

for (int i = 0; i < Gems.Length; i++)

{

// Pick a random spawn point

int randomIndex = Random.Range(0, availablePoints.Count);

Transform selectedPoint = availablePoints[randomIndex];

// Move the gem to that point

Gems[i].transform.position = selectedPoint.position;

Gems[i].transform.rotation = selectedPoint.rotation;

// Make sure the gem is active

Gems[i].SetActive(true);

// Remove the selected point so another gem can't use it

availablePoints.RemoveAt(randomIndex);

}

}

}

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Output:

The Puzzle Room scene was successfully created and tested in Unity. Interactive objects such as the treasure chest, gems, and puzzle elements were properly placed and arranged. The XR setup was configured successfully, and the objects could be interacted with using the XR Device Simulator during Play mode.



KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V PRACTICAL-8: Interactive UI

Aim

To create a **Main Menu and Gem Counter system in Unity** and connect the Main Menu with the Puzzle Room scene.

Theory:

Unity UI allows developers to create menus, buttons, and information displays for interactive applications. C# scripts can be used to control scene navigation and update UI elements dynamically. A gem counter can display the number of collected gems during gameplay.

Procedure:

Step-1: Create Main Menu UI: Open the Main Menu scene and add a **Canvas** from **UI**, then add the required raw image/background to the Canvas.

Step-2: Design Main Menu: Add and arrange the **Start Game** and **Exit Game** buttons on the Main Menu.

Step-3: Create Main Menu Script: Create a new C# script named **MainMenuManager** and open it for editing.

```
//MainMenuManager( script name )  
  
using UnityEngine;  
  
using UnityEngine.SceneManagement;  
  
public class MainMenuManager : MonoBehaviour  
{  
  
    public void StartGame()  
    {  
        SceneManager.LoadScene("Puzzle_Room");  
    }  
  
    public void ExitGame()  
    {  
        Application.Quit();  
    }  
}
```


KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Step-4: Add Scene Navigation Code: Add the `StartGame()` and `ExitGame()` functions to load the `Puzzle_Room` scene and exit the application.

Step-5: Add Puzzle Room: Add the `Puzzle Room` scene to the Hierarchy/scene setup so that it can be accessed from the Main Menu.

Step-6: Test Start Game: Run the Main Menu and click **Start Game** to verify that it takes the user to the Puzzle Room.

Step-7: Verify Scene Loading: Check that selecting **Start Game** successfully opens the Puzzle Room environment.

Step-8: Create Gem Counter Script: Create a new C# script named **GemCounterUI**.

```
//GemCounterUI
```

```
using TMPro;
```

```
using UnityEngine;
```

```
public class GemCounterUI : MonoBehaviour
```

```
{
```

```
    public TextMeshProUGUI counterText;
```

```
    public void UpdateCounter(int collected, int total)
```

```
    {
```

```
        counterText.text = "Gems: " + collected + "/" + total;
```

```
    }
```

```
}
```

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Step-9: Add Counter Code: Add the code to display the collected gems and total gems using TextMeshPro.

Step-10:Connect Counter UI: Add the script as a component and drag **GameCounter** from the Canvas in the Hierarchy into the required field in the Inspector.

Step-11:Create Gem Collection Manager: Create a new C# script named **GemCollectionManager**.

```
//GemCollectionManager
```

```
using UnityEngine;
```

```
public class GemCollectionManager : MonoBehaviour
```

```
{
```

```
    public int totalGems = 3;
```

```
    public int collectedGems = 0;
```

```
    public GemCounterUI counterUI;
```

```
    void Start()
```

```
    {
```

```
        UpdateUI();
```

```
    }
```

```
    public void GemPlaced()
```

```
    {
```

```
        collectedGems++;
```

```
        if (collectedGems > totalGems)
```

```
        {
```

```
            collectedGems = totalGems;
```

```
        }
```

```
        UpdateUI();
```

```
    }
```

```
    public void GemRemoved()
```

```
    {
```

```
        collectedGems--;
```

```
        if (collectedGems < 0)
```

```
        {
```

```
            collectedGems = 0;
```

```
        }
```

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

```
UpdateUI();  
}  
void UpdateUI()  
{  
    counterUI.UpdateCounter(collectedGems, totalGems);  
}  
}
```

Step-12: Add Collection Functions: Add the functions for updating the collected gem count, including **GemPlaced**, **GemRemoved**, and **UpdateUI**.

Step-13: Add Collection Manager: Add the **GemCollectionManager** script to the required GameObject.

Step-14: Configure Gem Sockets: Select the **three game sockets**, go to their Events section, add the required event options, and click + to configure the events.

Output: A fully functional main menu that loads the game environment when started, alongside an active HUD that updates dynamically (from 0/3 to 3/3) as the user places gems into the chest sockets





KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-9: VR Walkthrough

Aim:

To implement a **door-opening and game-completion system in Unity** using C# scripts, triggers, and UI buttons for restarting or exiting the game.

Theory:

Unity allows game events to be controlled using C# scripts and trigger components. In this practical, collecting all gems activates the door, while an exit trigger displays the completion UI. The completion UI provides options to restart the game or quit the application.

Procedure:

Step-1: Create Room and Door: Create a room and add the required door from the available prefab.

Step-2: Add Cutscene Camera: Add a camera to the scene and name it **CutSceneCamera**. Create the **DoorController** C# script and configure it with the left door, right door, and cutscene camera.

Step-3: Configure Door Opening: Add the door controller to the required GameObject and assign the left door, right door, and CutSceneCamera. Configure the script so that the doors open when the `OpenDoors()` function is called.

```
//DoorController
```

```
using UnityEngine;
```

```
public class DoorController : MonoBehaviour  
{
```

```
    public Transform leftDoor;  
    public Transform rightDoor;  
    public Camera cutsceneCamera;
```

```
    bool open = false;  
    float timer = 0;
```

```
    void Start()  
    {  
        cutsceneCamera.enabled = false;  
    }
```

```
    void Update()  
    {  
        if (open)  
        {  
            timer += Time.deltaTime;  
  
            leftDoor.localRotation = Quaternion.Lerp(  
                leftDoor.localRotation,  
                Quaternion.Euler(0, -90, 0),  
                Time.deltaTime * 2
```

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

```
);

rightDoor.localRotation = Quaternion.Lerp(
    rightDoor.localRotation,
    Quaternion.Euler(0, 90, 0),
    Time.deltaTime * 2
);

if (timer >= 3)
{
    cutsceneCamera.enabled = false;
}
}

public void OpenDoors()
{
    cutsceneCamera.enabled = true;
    open = true;
}
}
```

Step-4: Connect Gem Collection: Configure the **GCM** so that collecting the required gems triggers the door-opening function.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

Step-5: Create Exit Trigger: Create a new empty GameObject named **ExitTrigger** and create an **ExitTrigger** C# script. Add a **Box Collider**, enable **Is Trigger**, and attach the ExitTrigger script to the GameObject. Create a new game object as ExitTrigger

Create a new script as ExitTrigger

```
//ExitTrigger

using UnityEngine;

public class ExitTrigger : MonoBehaviour

{

    public GameObject gameCompletionUI;

    private void OnTriggerEnter(Collider other)

    {

        gameCompletionUI.SetActive(true);

    }

}
```

Step-6: Create Completion UI: Create an empty GameObject named **CompletionUI**. Add two UI buttons and name them **Restart Game** and **Quit**. Assign the CompletionUI to the ExitTrigger.

Step-7: Configure Game Completion: Create the **GameCompleteManager** script with Restart Game and Exit Game functions. Add the script to a new empty GameObject and configure the **Restart Game** button's **OnClick** event to call `GameCompleteManager.RestartGame()`.

Add the completion ui to the exittrigger

Step-8: Create a new script as the GameCompleteManager

```
//GameCompleteManager

using UnityEngine;

using UnityEngine.SceneManagement;

{

    public void RestartGame()

    {

        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);

    }

    public void ExitGame()
```

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

```
{  
    Application.Quit();  
}  
}
```

Step-9: Create the new empty in the hierarchy

And the script into it and then on the restart button onclick click plus and then add the object GameComplete and then GameCompleteManager.RestartGame

Output: A compiled, standalone VR application where the player can navigate the room, collect three gems to trigger a cinematic door opening, and step into an exit zone to reveal a functional restart/exit menu.





KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

PRACTICAL-10: Mini VR Experience

Aim:

To create an interactive 3D environment in Unity containing different objects such as crates, tables, and a cauldron.

To display relevant information about selected objects within the Unity scene.

Objectives

- To design and arrange a basic 3D environment using Unity objects and assets.
- To add interactive objects such as crates and a cauldron to the scene.
- To display object-related information such as weight, contents, physics, and status.
- To test the scene and verify that the required information is displayed correctly.

Procedure

Step-1: Open Unity Hub and create or open the required 3D Unity project.

Step-2: Create the basic 3D environment by adding objects such as walls, floor, tables, benches, and other required elements.

Step-3: Arrange the objects properly in the scene to create an organized 3D environment.

Step-4: Add interactive objects such as a wooden crate and a cauldron to the environment.

Step-5: Configure the crate object and provide its required properties, such as weight, contents, and physics information.

Step-6 :Configure the cauldron object and define its properties, including its status, contents, and last-used information.

Step-7: Add the required UI/information display so that information about an object can be shown when it is selected or interacted with.

Step-8: Test the crate interaction and verify that information such as **Weight: 10 kg, Contents: Simple Primitives, and Physics: Active** is displayed.

Step-9: Test the cauldron interaction and verify that information such as **Status: Empty, Contents: (Null), and Last used: (Null)** is displayed.

Step-10: Run the Unity scene and check that all objects, interactions, and information panels are working correctly.

Step-11: Save the Unity project and scene after confirming that the complete 3D environment and object information are displayed properly.

KISHINCHAND CHELLARAM COLLEGE, MUMBAI - 20

TYB.Sc (I.T.) Semester V

